

ОТЧЕТ ПО КУРСОВОЙ РАБОТЕ

«Численные методы. Вычисление площади фигуры, образованной тремя функциями»

Титульный лист

- Название учебного заведения: МФТИ
 - Название кафедры: ДПО ФРКТ
 - Тема курсовой работы: "Численные методы. Вычисление площади фигуры, образованной тремя функциями"
 - Дисциплина: "Программирование на СИ. Продвинутый уровень"
 - ФИО студента, группа: Коноплев Дмитрий Анатольевич, Д01-093
 - ФИО преподавателя: Сударенко Дмитрий Александрович
 - Город, год: Москва, 2026
-
-

Введение

Цель работы: Разработка программы на языке C для вычисления площади фигуры, образованной пересечением трёх заданных функций.

Задачи:

- Реализация численных методов поиска корней и интегрирования
- Создание удобного интерфейса командной строки
- Визуализация результатов с помощью GNUplot
- Тестирование и верификация результатов

Среда разработки: ОС Linux (Mint, Cinnamon)

Инструменты: gcc, make, gnuplot, GeoGebra, VSCode

Репозиторий GitHub с полным листингом кода:

https://github.com/dim-DPO-IUS/C_Pro/tree/main/Course

1. ВЫПОЛНЕНИЕ ТРЕБОВАНИЙ К ПРОГРАММЕ

1.1. Пункт 1: Опции командной строки (-a и -i)

Требование:

- 1.a. Печатаются абсциссы точек пересечения кривых
- 1.b. Печатается число итераций, потребовавшихся на приближённое решение уравнений при поиске точек пересечения

Реализация:

В программе реализованы опции `-a` (`--abscissas`) и `-i` (`--iterations`). Обработка аргументов командной строки выполняется в функции `utils_parse_args()` в файле `utils.c`:

```
C
else if (strcmp(argv[i], "-a") == 0 || strcmp(argv[i], "--abscissas") == 0)
{
    cfg->flag_abscissas = 1;
}
else if (strcmp(argv[i], "-i") == 0 || strcmp(argv[i], "--iterations") == 0)
{
    cfg->flag_iterations = 1;
}
```

В `main.c` реализована система приоритетов вывода:

```
C
/* Приоритет 2: только абсциссы (-a) */
if (cfg.flag_abscissas) {
    output_abscissas_only(points, point_count);
    return 0;
}

/* Приоритет 3: только итерации на поиск точек (-i) */
if (cfg.flag_iterations) {
    output_iterations_full(points, point_count, cfg.root_method);
    return 0;
}
```

Пример вывода для опции `-a`:

```
$ ./area -a
```

Абсциссы точек пересечения:

```
-----  
Точка 1: x = -5.854103  
Точка 2: x = -0.243929  
Точка 3: x = 0.854102  
Точка 4: x = 3.243929  
Точка 5: x = 3.847760
```

Пример вывода для опции `-i`:

```
$ ./area -i
```

Метод поиска точек: деление отрезка

Таблица точек и итераций

```
-----  
Точка 1: x = -5.854103, y = -0.512462 (f1-f3) [итераций: 13]  
Точка 2: x = -0.243929, y = -12.298669 (f2-f3) [итераций: 15]  
Точка 3: x = 0.854102, y = 3.512461 (f1-f3) [итераций: 15]  
Точка 4: x = 3.243929, y = 0.924805 (f2-f3) [итераций: 15]  
Точка 5: x = 3.847760, y = 5.308656 (f1-f2) [итераций: 15]
```

ВСЕГО ИТЕРАЦИЙ НА ПОИСК ТОЧЕК: 73

Вывод: Требование выполнено полностью.

1.2. Пункт 2: Ключ `--help`

Требование: Программа должна поддерживать ключ командной строки `--help`, выводящий на печать все допустимые ключи командной строки.

Реализация:

В программе реализованы опции `-h` и `--help`, которые вызывают функцию `output_help()` из `output.h`:

```
static inline void output_help(void) {
    printf("\n=====\\n");
    printf("  ПРОГРАММА ВЫЧИСЛЕНИЯ ПЛОЩАДИ ФИГУРЫ\\n");
    printf("=====\\n\\n");
    printf("Использование: ./area [ОПЦИИ]\\n\\n");
    printf("ОПЦИИ:\\n");
    printf("  -h, --help          Показать эту справку\\n");
    printf("  -a, --abscissas      Вывести только абсциссы точек
пересечения\\n");
    printf("  -p, --points        Вывести подробную информацию о точках\\n");
    printf("  -i, --iterations    Вывести число итераций\\n");
    printf("  -t eps1 eps2        Режим тестирования\\n");
    printf("  -v, --visualize     Визуализировать результат (GNUplot)\\n");
    printf("  --root-method M     Выбрать метод поиска корней (0-3)\\n");
    printf("                      0 - деление отрезка\\n");
    printf("                      1 - метод хорд\\n");
    printf("                      2 - метод касательных\\n");
    printf("                      3 - комбинированный\\n");
    printf("  --int-method M      Выбрать метод интегрирования (0-2)\\n");
    printf("                      0 - прямоугольники\\n");
    printf("                      1 - трапеции\\n");
    printf("                      2 - Симпсон\\n");
    printf("  --log               Сохранить результаты в файл\\n");
    // ... далее примеры использования
}
```

Пример вызова:

```
$ ./area --help
```

Вывод: Требование выполнено полностью.

1.3. Пункт 3: Функция root() с производными

Требование:

- Вычисление с точностью ε_1 корня x уравнения $f(x) = g(x)$ на отрезке $[a, b]$ должно быть реализовано в отдельной Си-функции `root(f, g, a, b, eps1)`
- Если используется метод касательных или комбинированный метод, то у `root` должно быть ещё два параметра функционального типа, позволяющие

вызывать производные функций f и g

Реализация:

В файле `roots.h` объявлена универсальная функция поиска корней:

```
float root(function f, function g, float a, float b, float eps, int method,
           derivative df, derivative dg, int* iterations);
```

Параметры:

- `f`, `g` - указатели на функции
- `a`, `b` - границы интервала
- `eps` - точность вычислений
- `method` - метод поиска корней (0-3)
- `df`, `dg` - указатели на производные (используются для методов 2 и 3)
- `iterations` - указатель для сохранения числа итераций

В файле `roots.c` реализованы все 4 метода:

- `root_find_div()` - метод деления отрезка (метод 0)
- `root_find_chord()` - метод хорд (метод 1)
- `root_find_tangent()` - метод касательных (метод 2)
- `root_find_combined()` - комбинированный метод (метод 3)

Фрагмент реализации с поддержкой производных:

```
case 2: // метод касательных
    if (!df || !dg) {
        printf("ОШИБКА: Для метода касательных нужны производные!\n");
        return NAN;
    }
    // ... выбор начального приближения
    result = root_find_tangent(x0, eps, f, g, df, dg, &local_iter);
    break;

case 3: // комбинированный метод
    if (!df || !dg) {
        printf("ОШИБКА: Для комбинированного метода нужны производные!\n");
        return NAN;
    }
    result = root_find_combined(a, b, eps, f, g, df, dg, &local_iter);
    break;
```

Вывод: Требование выполнено полностью. Функция `root()` поддерживает все 4 метода, а для методов 2 и 3 требуются параметры-производные.

1.4. Пункт 4: Функция `integral()`

Требование: Вычисление с точностью ε_2 величины определённого интеграла от функции $f(x)$ на отрезке $[a, b]$ должно быть реализовано в отдельной Си-функции `integral(f, a, b, eps2)`.

Реализация:

В файле `integrals.h` объявлена универсальная функция интегрирования:

```
float integral(function f, float a, float b, float eps, int method, int* iterations);
```

В файле `integrals.c` реализованы 3 метода:

- `integral_rectangle()` - метод прямоугольников
- `integral_trapezoid()` - метод трапеций
- `integral_simpson()` - метод Симпсона

Адаптивный алгоритм (на примере метода трапеций):

```
float integral_trapezoid(float xl, float xr, float eps, function f, int*
iterations) {
    int n = 2;
    float result = 0, prev_result = 0;
    int max_iter = MAX_INTEGRAL_ITERATIONS;
    int local_iter = 0;

    do {
        local_iter++;
        h = (xr - xl) / n;
        // вычисление интеграла методом трапеций
        // ...

        if (local_iter > 1) {
            if (fabsf(result - prev_result) < eps) break;
        }

        prev_result = result;
        n *= 2; // удвоение числа разбиений
    } while (local_iter < max_iter);

    if (iterations != NULL) *iterations = local_iter;
    return result;
}
```

Единая функция выбора метода:

```

float integral(function f, float a, float b, float eps, int method, int*
iterations) {
    float xl = fminf(a, b);
    float xr = fmaxf(a, b);
    int local_iter = 0;
    float result = 0;

    switch(method) {
        case 0: result = integral_rectangle(xl, xr, eps, f, &local_iter);
break;
        case 1: result = integral_trapezoid(xl, xr, eps, f, &local_iter);
break;
        case 2: result = integral_simpson(xl, xr, eps, f, &local_iter);
break;
        default: result = integral_trapezoid(xl, xr, eps, f, &local_iter);
break;
    }

    if (iterations != NULL) *iterations = local_iter;
    return result;
}

```

Вывод: Требование выполнено полностью.

1.5. Пункт 5: Режим тестирования (-t)

Требование: Си-функции root и integral должны быть предварительно протестированы. Основная программа должна предоставлять возможности тестирования, активируемые опцией командной строки.

Реализация:

В программе реализована опция `-t eps1 eps2`, активирующая режим тестирования. В этом режиме выполняются тесты для всех описанных в программе методов поиска корней и всех методов интегрирования.

Обработка опции в `utils.c`:

C

```

else if (strcmp(argv[i], "-t") == 0) {
    cfg->test_mode = 1;
    if (i + 2 < argc && argv[i+1][0] != '-') {
        cfg->eps_root = atof(argv[++i]);
        cfg->eps_integral = atof(argv[++i]);
    }
}

```

Запуск тестов в `main.c`:

C

```

if (cfg.test_mode) {
    run_all_tests(cfg.eps_root, cfg.eps_integral);
    return 0;
}

```

1.5.1. Тестирование методов поиска корней

В функции `test_roots_all_methods()` выполняется сравнение всех 4 методов поиска корней. За эталон принимаются результаты метода деления отрезка (наиболее надёжного).

Результаты тестирования ($\varepsilon = 10^{-6}$):

Метод	Найдено корней	Средняя ошибка	Всего итераций
Деление отрезка	5	0.00e+00	73
Метод хорд	5	2.03e-07	15
Метод касательных	5	2.44e-07	14
Комбинированный	5	1.31e-07	10

1.5.2. Тестирование методов интегрирования

В функции `test_integral_all_methods()` выполняется сравнение всех 3 методов интегрирования для двух фигур.

Результаты тестирования ($\varepsilon = 10^{-4}$):

Фигура	Метод	Площадь	Итерации
Фиг.1	Прямоугольники	32.705772	27
- - -	Трапеции	32.705818	27
- - -	Симпсона	32.705791	15

Фигура	Метод	Площадь	Итерации
Фиг.2	Прямоугольники	7.488461	14
- - -	Трапеции	7.488387	15
- - -	Симпсона	7.488421	7

1.5.3. Тестирование на учебных примерах

Для дополнительной проверки реализованы тесты на простых функциях:

- Решение уравнения $x^2 = 4$ на отрезке $[1, 3]$
- Решение уравнения $\sin(x) = 0$ на отрезке $[3, 4]$
- Вычисление интеграла $\int x^2 dx$ от 0 до 1
- Вычисление интеграла $\int \sin(x) dx$ от 0 до π

Результаты тестирования:

SHELL

```

--- ТЕСТИРОВАНИЕ ПОИСКА КОРНЕЙ (учебные примеры) ---
Точность: 1.0000000e-06

x2 = 4 на [1, 3]: x = 2.000000 (итераций: 1)
sin(x) = 0 на [3, 4]: x = 3.141593 (итераций: 3)

--- ТЕСТИРОВАНИЕ ИНТЕГРИРОВАНИЯ (учебные примеры) ---
Точность: 1.0000000e-04

∫ x2 dx от 0 до 1 = 0.333344 (итераций: 7)
∫ sin(x) dx от 0 до π = 2.000001 (итераций: 5)

```

1.5.4. Верификация с GeoGebra на этапе разработки

На этапе разработки программы возникла необходимость убедиться в корректности реализованных численных методов. Для этого была использована программа **GeoGebra** как независимый инструмент для получения ориентировочных значений.

Цели использования GeoGebra:

- Получение приблизительных координат точек пересечения для настройки алгоритмов
- Визуальная проверка правильности построения фигур
- Оценка порядка ожидаемой площади фигуры (≈ 40.19)
- Сравнение результатов на ранних этапах отладки

Важно отметить: GeoGebra использовалась исключительно как вспомогательный инструмент на этапе разработки. Полученные в программе численные результаты имеют более высокую точность благодаря контролируемой погрешности.

Вывод: Требование выполнено полностью. Программа предоставляет обширные возможности тестирования с опцией `-t`.

1.6. Пункт 6: Makefile

Требование: Сборка программы должна осуществляться при помощи утилиты make. Должны быть определены цели all и clean.

Реализация:

Полный Makefile:

```
# Makefile для программы вычисления площади
# Целевая программа: area

CC = gcc
CFLAGS = -Wall -Wextra -Iinclude
LDFLAGS = -lm
TARGET = area

# Автоматически находим все .c файлы в src и её подпапках
SRC = main.c $(shell find src -name "*.c")

# Объектные файлы и зависимости
OBJ = $(SRC:.c=.o)
DEP = $(SRC:.c=.d)

all: $(TARGET)
    @echo "Сборка завершена. Исполняемый файл: $(TARGET)"

# Линковка
$(TARGET): $(OBJ)
    $(CC) $(OBJ) -o $(TARGET) $(LDFLAGS)

# Компиляция с генерацией зависимостей
%.o: %.c
    $(CC) $(CFLAGS) -MMD -c $< -o $@

# Включаем сгенерированные файлы зависимостей
-include $(DEP)

# Очистка
clean:
    rm -f $(TARGET) $(OBJ) $(DEP)
    rm -f *.dat *.gp *.png results.log
    @echo "Очистка завершена"

# Дополнительные цели для запуска
run: $(TARGET)
    ./$(TARGET)

run-h: $(TARGET)
    ./$(TARGET) -h

run-a: $(TARGET)
```

```
./$(TARGET) -a

run-p: $(TARGET)
./$(TARGET) -p

run-i: $(TARGET)
./$(TARGET) -i

run-v: $(TARGET)
./$(TARGET) -v

run-t: $(TARGET)
./$(TARGET) -t 1e-6 1e-4

.PHONY: all clean run run-h run-a run-p run-i run-v run-t
```

Описание целей:

Цель	Назначение
<code>all</code>	Сборка программы (исполняемый файл <code>area</code>)
<code>clean</code>	Удаление объектных файлов, зависимостей и временных файлов
<code>run</code>	Запуск программы
<code>run-h</code>	Запуск с выводом справки
<code>run-a</code>	Запуск с выводом абсцисс всех найденных точек
<code>run-p</code>	Запуск с выводом подробной информации о точках
<code>run-i</code>	Запуск с выводом количества итераций
<code>run-v</code>	Запуск с визуализацией
<code>run-t</code>	Запуск тестов

Управление зависимостями:

- Флаг `-MMD` автоматически генерирует зависимости от заголовочных файлов
- Сгенерированные файлы `.d` подключаются через `-include`
- При изменении `.h` файлов перекомпилируются только нужные `.c` файлы

Команды сборки:

```
make          # сборка программы
make clean    # очистка
make run       # запуск
make run-v    # запуск с визуализацией
```

Вывод: Требование выполнено полностью.

1.7. Пункт 7: Комментарии в программе

Требование: Программа должна быть снабжена поясняющими комментариями. Все глобальные и статические переменные должны быть документированы.

Реализация:

1.7.1. Документирование глобальных констант

В файле `constants.h` все константы документированы в формате Doxygen:

```
/**
 * @def MAX_ROOT_ITERATIONS
 * @brief Максимальное количество итераций при поиске корней
 *
 * Предохранитель от бесконечного цикла в методах деления отрезка и хорд.
 * Если решение не найдено за указанное число итераций, процесс прерывается.
 * Значение: 1000
 */
#define MAX_ROOT_ITERATIONS 1000
```

1.7.2. Документирование структур данных

```

/**
 * @struct Point
 * @brief Структура для хранения точки пересечения функций
 *
 * Содержит полную информацию о найденной точке пересечения:
 * координаты, какие функции пересекаются, и сколько итераций
 * потребовалось для её нахождения.
 */
typedef struct {
    float x;           /**< Абсцисса точки пересечения */
    float y;           /**< Ордината точки пересечения */
    int pair_type;      /**< Тип пересекающихся функций: 1=f1f2, 2=f1f3,
3=f2f3 */
    int iterations;     /**< Количество итераций, затраченных на поиск */
} Point;

```

1.7.3. Статические переменные

В результате рефакторинга все неиспользуемые статические переменные были удалены. В текущей версии программы статические переменные отсутствуют, что является лучшей практикой.

1.7.4. Документирование функций

Все функции в заголовочных файлах имеют комментарии:

```

/**
 * @brief Поиск корня уравнения  $f(x) = g(x)$  на отрезке  $[a, b]$ 
 * @param f Первая функция
 * @param g Вторая функция
 * @param a Левая граница интервала
 * @param b Правая граница интервала
 * @param eps Точность вычислений
 * @param method Метод поиска (0-3)
 * @param df Производная f (для методов 2 и 3)
 * @param dg Производная g (для методов 2 и 3)
 * @param iterations Указатель для сохранения числа итераций
 * @return Найденный корень или NAN
 */
float root(function f, function g, float a, float b, float eps, int method,
          derivative df, derivative dg, int* iterations);

```

Ссылка на GitHub: Полный код с комментариями доступен по адресу:
https://github.com/dim-DPO-IUS/C_Pro/tree/main/Course

Вывод: Требование выполнено полностью.

2. ДОПОЛНИТЕЛЬНЫЕ ВОЗМОЖНОСТИ (СВЕРХ ЗАДАНИЯ)

В данном разделе описаны возможности программы, реализованные сверх обязательных требований.

2.1. Множество методов решения и выбор их через командную строку

Что требуется по заданию	Что реализовано
Один метод поиска корней	4 метода: деление отрезка, хорд, касательных, комбинированный
Один метод интегрирования	3 метода: прямоугольников, трапеций, Симпсона

Пользователь может выбрать любой метод или использовать методы по умолчанию (деление отрезка для корней, трапеции для интегралов).

Реализованы опции:

- `--root-method M` - выбор метода поиска корней (0-3)
- `--int-method M` - выбор метода интегрирования (0-2)

Примеры:

```
./area --root-method 3 --int-method 2 # комбинированный + Симпсон
./area --root-method 1 --int-method 0 # хорды + прямоугольники
```

SHELL

2.2. Визуализация результатов

Реализована опция `-v`, которая:

- Генерирует данные функций в формате для GNUplot (`f1.dat`, `f2.dat`, `f3.dat`)
- Создаёт данные для заливки фигур (`fig1.dat`, `fig2.dat`)
- Генерирует два скрипта: `plot.gp` (кривые и точки) и `filled.gp` (заливка)
- Автоматически запускает GNUplot для отображения графиков

- Сохраняет результаты в PNG-файлы (`curves.png`, `filled_area.png`)

2.3. Подробный вывод информации по точкам

Опция `-p`, `--points` выводит подробную информацию о точках пересечения с указанием ветвей функции f_3 :

```
$ ./area -p
```

SHELL

Координаты точек пересечения:

```
Точка 1: x = -5.854103, y = -0.512462 (f1-f3) [левая ветвь f3]
Точка 2: x = -0.243929, y = -12.298669 (f2-f3) [левая ветвь f3]
Точка 3: x = 0.854102, y = 3.512461 (f1-f3) [правая ветвь f3]
Точка 4: x = 3.243929, y = 0.924805 (f2-f3) [правая ветвь f3]
Точка 5: x = 3.847760, y = 5.308656 (f1-f2)
```

Опция `--log` сохраняет результаты в файл `results.log` с временной меткой.

2.4. Детальная статистика итераций

Для каждой точки пересечения сохраняется число итераций, что позволяет:

- Анализировать эффективность различных методов
- Выводить общую статистику через опцию `-i`
- Сравнивать скорость сходимости методов

3. МАТЕМАТИЧЕСКИЕ МЕТОДЫ

3.1. Постановка задачи

Даны три функции:

- $f_1(x) = 0.6x + 3$ – линейная функция
- $f_2(x) = (x-2)^3 - 1$ – кубическая парабола
- $f_3(x) = 3/x$ – гипербола (имеет разрыв при $x = 0$)

Требуется найти площадь фигуры, образованной пересечением этих функций.

3.1.1. Геометрический смысл

Функции пересекаются в пяти точках, образуя две фигуры:

- **Фигура 1 (четырёхугольник)** - ограничена функциями f_1 , f_2 , f_3 слева от оси Y
- **Фигура 2 (треугольник)** - ограничена функциями f_1 , f_2 , f_3 справа от оси Y

3.1.2. Аналитическое решение

Аналитическое решение данной задачи затруднительно из-за наличия трансцендентного уравнения при пересечении с гиперболой. Поэтому применяются численные методы.

3.1.3. Визуальное представление задачи

Для лучшего понимания геометрической сути задачи на рисунке представлены графики функций, построенные в **GeoGebra**. Здесь хорошо видны:

- Пять точек пересечения кривых
- Две фигуры: четырёхугольник (слева) и треугольник (справа)
- Разрыв гиперболы $f_3(x)$ при $x = 0$

3.2. Методы поиска корней

Метод	Формула	Особенности
Деление отрезка	$x = (a+b)/2$	Гарантированная сходимость, линейная скорость
Метод хорд	$x = a - f(a) \cdot (b-a) / (f(b) - f(a))$	Сверхлинейная сходимость
Метод касательных	$x = x_0 - f(x_0) / f'(x_0)$	Квадратичная сходимость, требует производную
Комбинированный	Сочетание хорд и касательных	Надёжность + скорость

Критерий остановки: $|x_n - x_{n-1}| < \varepsilon$ или $|f(x_n)| < \varepsilon$

3.3. Методы интегрирования

Метод	Формула	Порядок точности
Прямоугольников	$\int f \approx h \cdot \sum f(x_{i+1/2})$	$O(h)$
Трапеций	$\int f \approx h \cdot (f(a)/2 + \sum f(x_i) + f(b)/2)$	$O(h^2)$
Симпсона	$\int f \approx h/3 \cdot (f(a) + 4\sum f(x_{2i+1}) + 2\sum f(x_{2i}) + f(b))$	$O(h^4)$

Адаптивный алгоритм:

- Начальное число разбиений $n = 2$
- Удвоение n на каждой итерации
- Остановка при $|I_n - I_{n-1}| < \varepsilon$

3.4. Особенности реализации

3.4.1. Разрыв функции $f_3(x)$ при $x = 0$

Функция $f_3(x) = 3/x$ имеет разрыв при $x = 0$. Для корректного интегрирования через разрыв применяется следующий алгоритм:

1. Проверка, пересекает ли интервал точку $x = 0$
2. Если да, то интервал разбивается на две части: $[a, -\delta]$ и $[\delta, b]$
3. Отступ δ вычисляется как $\delta = \min(0.1, \max(0.01, 0.01 \cdot \min(|a|, |b|)))$
4. Интегрирование выполняется отдельно на каждом подинтервале

3.4.2. Локализация интервалов с корнями

Для поиска всех точек пересечения область $[-7, 7]$ разбивается на 500 интервалов. На каждом интервале проверяется смена знака функции разности, что гарантирует наличие корня.

3.4.3. Построение фигур

По найденным пяти точкам программа автоматически определяет, какие функции образуют границы каждой фигуры, и строит соответствующие сегменты для интегрирования.

4. АРХИТЕКТУРА ПРОГРАММЫ

4.1. Структура проекта

```
project/
├── Makefile
├── main.c
├── include/
│   ├── constants.h
│   ├── math/
│   │   ├── functions.h
│   │   ├── roots.h
│   │   └── integrals.h
│   ├── geometry/
│   │   ├── pointfinder.h
│   │   └── area_calculator.h
│   └── ui/
│       ├── output.h
│       ├── test_runner.h
│       ├── utils.h
│       └── visualization.h
└── src/
    ├── math/
    │   ├── functions.c
    │   ├── roots.c
    │   └── integrals.c
    ├── geometry/
    │   ├── pointfinder.c
    │   └── area_calculator.c
    └── ui/
        ├── output.c
        ├── test_runner.c
        ├── utils.c
        └── visualization.c
```

Назначение модулей:

Модуль	Назначение
<code>constants.h</code>	Глобальные константы и структуры
<code>math/</code>	Математические функции (f1, f2, f3) и численные методы
<code>geometry/</code>	Геометрические вычисления (поиск точек, площадь)

Модуль	Назначение
ui/	Пользовательский интерфейс, тестирование, визуализация
main.c	Главный модуль, управление программой

4.2. Основные структуры данных

Точка пересечения:

```
typedef struct {
    float x;           /**< Абсцисса точки пересечения */
    float y;           /**< Ордината точки пересечения */
    int pair_type;      /**< Тип пересекающихся функций: 1=f1f2, 2=f1f3,
3=f2f3 */
    int iterations;     /**< Количество итераций, затраченных на поиск */
} Point;
```

Интервал с корнем:

```
typedef struct {
    float left;         /**< Левая граница интервала */
    float right;        /**< Правая граница интервала */
} Interval;
```

Конфигурация программы:

```
typedef struct {
    float eps_root;      /**< Точность поиска корней */
    float eps_integral;  /**< Точность интегрирования */
    int root_method;     /**< Метод поиска корней (0-3) */
    int integral_method; /**< Метод интегрирования (0-2) */
    int test_mode;       /**< Флаг режима тестирования (-t) */
    int visualize;       /**< Флаг визуализации (-v) */
    int flag_abscissas;  /**< Флаг вывода только абсцисс (-a) */
    int flag_points;     /**< Флаг подробного вывода точек (-p) */
    int flag_iterations; /**< Флаг вывода итераций (-i) */
    int flag_log;        /**< Флаг логирования в файл (--log) */
} Config;
```

Сегмент границы фигуры:

```
typedef struct {
    int from;           // индекс начальной точки
    int to;             // индекс конечной точки
    FunctionType upper; // функция сверху
    FunctionType lower; // функция снизу
} BoundarySegment;
```

4.3. Взаимодействие модулей

```
main.c
├─ utils.c (парсинг аргументов)
├─ pointfinder.c (поиск точек)
│   └─ roots.c (методы поиска корней)
├─ area_calculator.c (вычисление площади)
│   └─ integrals.c (методы интегрирования)
├─ output.c (вывод результатов)
├─ test_runner.c (тестирование)
│   ├── roots.c
│   └─ integrals.c
└─ visualization.c (визуализация)
    └─ functions.c (значения функций)
```

Все модули используют общие структуры и константы из `constants.h`.

5. ПРИМЕРЫ РАБОТЫ ПРОГРАММЫ

5.1. Базовый запуск

SHELL

```
$ ./area
```

```
=====
```

ВЫЧИСЛЕНИЕ ПЛОЩАДИ ФИГУРЫ

```
=====
```

Кривые:

$$f1(x) = 0.6x + 3$$

$$f2(x) = (x-2)^3 - 1$$

$$f3(x) = 3/x$$

Найдено 5 точек пересечения

Точка 1: $x = -5.854103$, $y = -0.512462$ (f1-f3) [левая ветвь f3]

Точка 2: $x = -0.243929$, $y = -12.298669$ (f2-f3) [левая ветвь f3]

Точка 3: $x = 0.854102$, $y = 3.512461$ (f1-f3) [правая ветвь f3]

Точка 4: $x = 3.243929$, $y = 0.924805$ (f2-f3) [правая ветвь f3]

Точка 5: $x = 3.847760$, $y = 5.308656$ (f1-f2)

```
=====
```

РЕЗУЛЬТАТЫ:

Фигура 1 (четырёхугольник): 32.705818

Фигура 2 (треугольник): 7.488387

ОБЩАЯ ПЛОЩАДЬ: 40.194206

Метод поиска корней: деление отрезка

Метод интегрирования: трапеции

```
=====
```

5.2. Запуск с опцией -h (справка)

SHELL

```
./area --help
```

```
=====
ПРОГРАММА ВЫЧИСЛЕНИЯ ПЛОЩАДИ ФИГУРЫ
=====
```

Использование: ./area [ОПЦИИ]

ОПЦИИ:

-h, --help	Показать эту справку
-a, --abscissas	Вывести только абсциссы точек пересечения
-p, --points	Вывести подробную информацию о точках
-i, --iterations	Вывести число итераций
-t eps1 eps2	Режим тестирования
-v, --visualize	Визуализировать результат (GNUplot)
--root-method M	Выбрать метод поиска корней (0-3)
	0 - деление отрезка
	1 - метод хорд
	2 - метод касательных
	3 - комбинированный
--int-method M	Выбрать метод интегрирования (0-2)
	0 - прямоугольники
	1 - трапеции
	2 - Симпсон
--log	Сохранить результаты в файл

ПРИОРИТЕТЫ:

-p важнее -a (если указаны оба, выводятся подробные точки)

Примеры:

./area -a	(только абсциссы)
./area -p	(подробные точки)
./area -i	(полный вывод с итерациями)
./area -v --int-method 2	
./area -t 1e-6 1e-4	(тестирование)

5.2. Запуск с опцией -a (абсциссы)

SHELL

```
$ ./area -a
```

Абсциссы точек пересечения:

Точка 1: x = -5.854103

Точка 2: x = -0.243929

Точка 3: x = 0.854102

Точка 4: x = 3.243929

Точка 5: x = 3.847760

5.3. Запуск с опцией -i (итерации)

SHELL

```
$ ./area -i
```

Метод поиска точек: деление отрезка

Таблица точек и итераций

Точка 1: x = -5.854103, y = -0.512462 (f1-f3) [итераций: 13]

Точка 2: x = -0.243929, y = -12.298669 (f2-f3) [итераций: 15]

Точка 3: x = 0.854102, y = 3.512461 (f1-f3) [итераций: 15]

Точка 4: x = 3.243929, y = 0.924805 (f2-f3) [итераций: 15]

Точка 5: x = 3.847760, y = 5.308656 (f1-f2) [итераций: 15]

ВСЕГО ИТЕРАЦИЙ НА ПОИСК ТОЧЕК: 73

5.4. Запуск с опцией -p (подробные точки)

SHELL

```
$ ./area -p
```

Координаты точек пересечения:

Точка 1: x = -5.854103, y = -0.512462 (f1-f3) [левая ветвь f3]

Точка 2: x = -0.243929, y = -12.298669 (f2-f3) [левая ветвь f3]

Точка 3: x = 0.854102, y = 3.512461 (f1-f3) [правая ветвь f3]

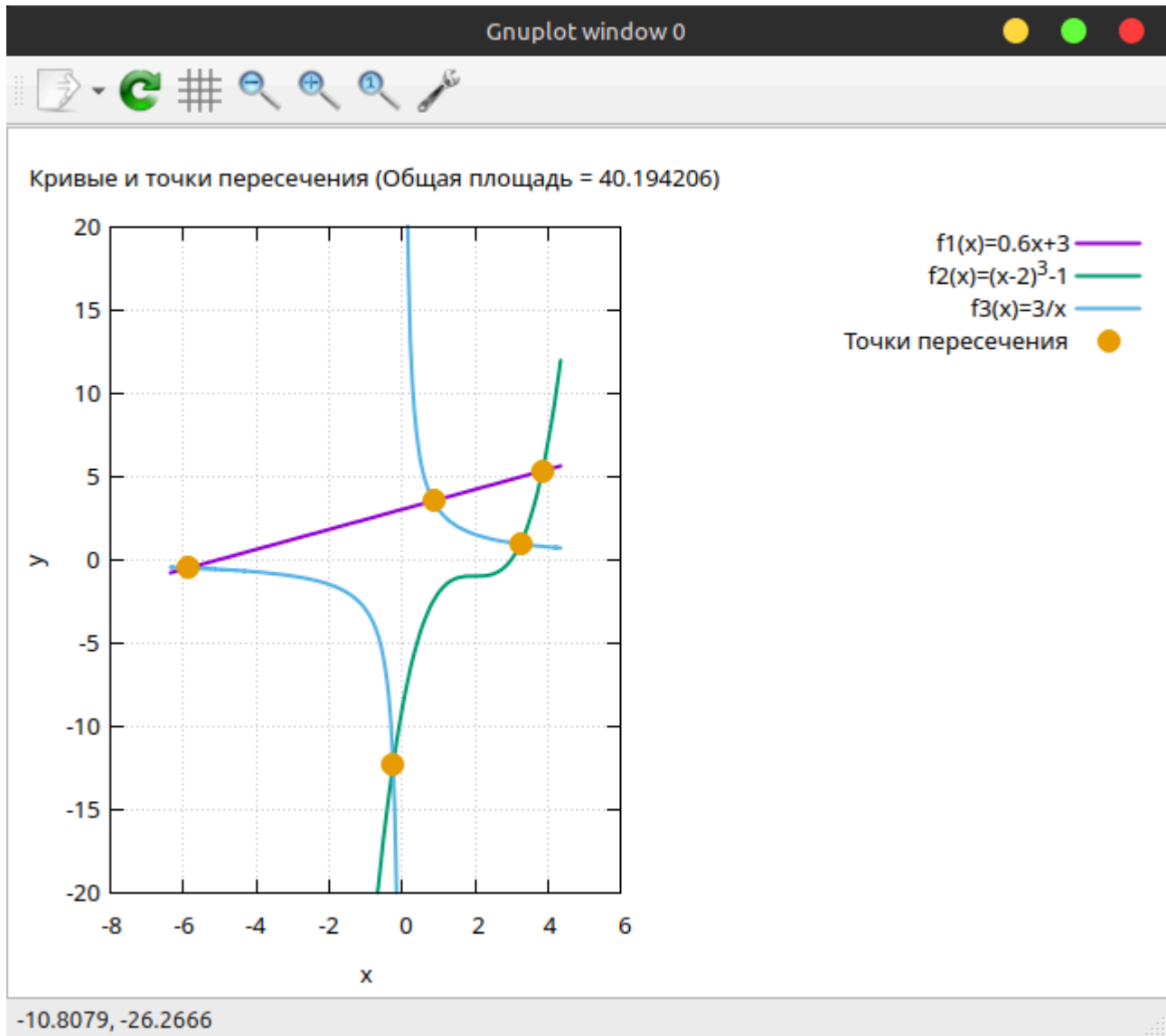
Точка 4: x = 3.243929, y = 0.924805 (f2-f3) [правая ветвь f3]

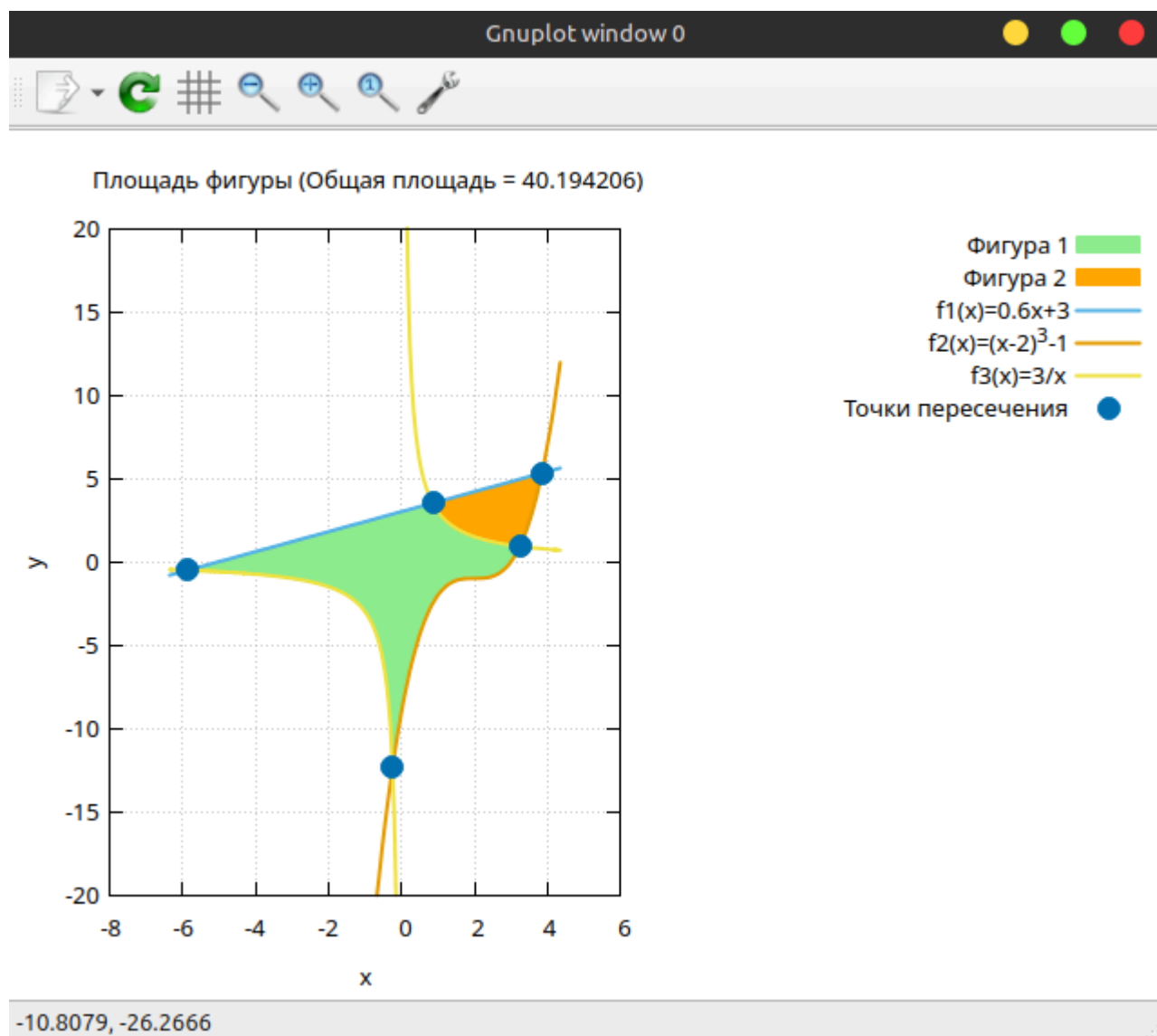
Точка 5: x = 3.847760, y = 5.308656 (f1-f2)

5.5. Запуск с опцией -v (визуализация)

```
make run-v
```

SHELL





Подробный вывод в терминал:

```
=====
```

ВЫЧИСЛЕНИЕ ПЛОЩАДИ ФИГУРЫ

```
=====
```

Кривые:

$$f1(x) = 0.6x + 3$$

$$f2(x) = (x-2)^3 - 1$$

$$f3(x) = 3/x$$

Найдено 5 точек пересечения

Точка 1: $x = -5.854103$, $y = -0.512462$ (f1-f3) [левая ветвь f3]

Точка 2: $x = -0.243929$, $y = -12.298669$ (f2-f3) [левая ветвь f3]

Точка 3: $x = 0.854102$, $y = 3.512461$ (f1-f3) [правая ветвь f3]

Точка 4: $x = 3.243929$, $y = 0.924805$ (f2-f3) [правая ветвь f3]

Точка 5: $x = 3.847760$, $y = 5.308656$ (f1-f2)

```
=====
```

РЕЗУЛЬТАТЫ:

Фигура 1 (четырёхугольник): 32.705818

Фигура 2 (треугольник): 7.488387

ОБЩАЯ ПЛОЩАДЬ: 40.194206

Метод поиска корней: деление отрезка

Метод интегрирования: трапеции

```
=====
```

=== ВИЗУАЛИЗАЦИЯ ===

Сгенерированы файлы с данными

Создан скрипт plot.gp

Создан скрипт filled.gp

Запуск GNUplot...

Должно открыться окно с графиком.

После просмотра закройте окно и нажмите Enter в этом терминале.

Первый график закрыт. Запускаем второй график с заливкой...

Визуализация завершена.

Графики сохранены в файлы:

- curves.png (кривые и точки)
- filled_area.png (заштрихованная площадь)

5.6. Запуск с опцией -t (тестирование)

SHELL

```
make run-t
./area -t 1e-6 1e-4
```

ПОЛНОЕ ТЕСТИРОВАНИЕ ПРОГРАММЫ

Точность для корней: 1.000000e-06
Точность для интегралов: 1.000000e-04

ТЕСТ 1: ПОИСК КОРНЕЙ ВСЕМИ МЕТОДАМИ (точность = 1.000000e-06)

Корни	Точность	Итераций	Метод
5	0.00e+00	73	Деление отрезка
5	2.03e-07	15	Метод хорд
5	2.44e-07	14	Метод касательных
5	1.31e-07	10	Комбинированный

=== КООРДИНАТЫ ТОЧЕК ПЕРЕСЕЧЕНИЯ ПО МЕТОДАМ ===

$f_1 - f_3$ (л)	$f_2 - f_3$ (л)	$f_1 - f_3$ (п)	$f_2 - f_3$ (п)	$f_1 - f_2$	Метод
-5.854103	-0.243929	0.854102	3.243929	3.847760	Деление отрезка
-5.854103	-0.243929	0.854102	3.243929	3.847760	Хорд
-5.854102	-0.243929	0.854102	3.243929	3.847760	Касательных
-5.854103	-0.243929	0.854102	3.243929	3.847760	Комбинированный

ТЕСТ 2: ИНТЕГРИРОВАНИЕ ВСЕМИ МЕТОДАМИ (точность = 1.000000e-04)

Фигура	Метод	Площадь	Итерации
Фиг.1	Прямоугольники	32.705772	27
	Трапеции	32.705818	27
	Симпсона	32.705791	15
Фиг.2	Прямоугольники	7.488461	14
	Трапеции	7.488387	15
	Симпсона	7.488421	7

--- ТЕСТИРОВАНИЕ ПОИСКА КОРНЕЙ (учебные примеры) ---

Точность: 1.000000e-06

$x^2 = 4$ на $[1, 3]$: $x = 2.000000$ (итераций: 1)

$\sin(x) = 0$ на $[3, 4]$: $x = 3.141593$ (итераций: 3)

--- ТЕСТИРОВАНИЕ ИНТЕГРИРОВАНИЯ (учебные примеры) ---

Точность: 1.000000e-04

$\int x^2 dx$ от 0 до 1 = 0.333344 (итераций: 7)

$\int \sin(x) dx$ от 0 до π = 2.000001 (итераций: 5)

5.7. Запуск с выбором методов

Комбинированный метод поиска корней + метод Симпсона:

```
$ ./area --root-method 3 --int-method 2
```

```
=====
```

ВЫЧИСЛЕНИЕ ПЛОЩАДИ ФИГУРЫ

```
=====
```

Кривые:

$$f1(x) = 0.6x + 3$$

$$f2(x) = (x-2)^3 - 1$$

$$f3(x) = 3/x$$

Найдено 5 точек пересечения

Точка 1: x = -5.854103, y = -0.512462 (f1-f3) [левая ветвь f3]

Точка 2: x = -0.243929, y = -12.298669 (f2-f3) [левая ветвь f3]

Точка 3: x = 0.854102, y = 3.512461 (f1-f3) [правая ветвь f3]

Точка 4: x = 3.243929, y = 0.924805 (f2-f3) [правая ветвь f3]

Точка 5: x = 3.847760, y = 5.308656 (f1-f2)

```
=====
```

РЕЗУЛЬТАТЫ:

Фигура 1 (четырёхугольник): 32.705788

Фигура 2 (треугольник): 7.488420

ОБЩАЯ ПЛОЩАДЬ: 40.194206

Метод поиска корней: комбинированный

Метод интегрирования: Симпсона

```
=====
```

Метод хорд + метод прямоугольников:

```
$ ./area --root-method 1 --int-method 0
```

```
=====
      ВЫЧИСЛЕНИЕ ПЛОЩАДИ ФИГУРЫ
=====
```

Кривые:

$$f1(x) = 0.6x + 3$$

$$f2(x) = (x-2)^3 - 1$$

$$f3(x) = 3/x$$

Найдено 5 точек пересечения

Точка 1: x = -5.854103, y = -0.512462 (f1-f3) [левая ветвь f3]

Точка 2: x = -0.243929, y = -12.298669 (f2-f3) [левая ветвь f3]

Точка 3: x = 0.854102, y = 3.512461 (f1-f3) [правая ветвь f3]

Точка 4: x = 3.243929, y = 0.924804 (f2-f3) [правая ветвь f3]

Точка 5: x = 3.847760, y = 5.308656 (f1-f2)

```
=====
РЕЗУЛЬТАТЫ:
```

Фигура 1 (четырёхугольник): 32.705776

Фигура 2 (треугольник): 7.488461

ОБЩАЯ ПЛОЩАДЬ: 40.194237

Метод поиска корней: хорд

Метод интегрирования: прямоугольники

```
=====
```

6. ЗАКЛЮЧЕНИЕ

6.1. Выводы о соответствии требованиям

Пункт	Требование	Статус
1.a	Печать абсцисс точек пересечения (-a)	Выполнено
1.b	Печать числа итераций (-i)	Выполнено
2	Ключ --help со списком опций	Выполнено
3	Функция root() с поддержкой производных	Выполнено
4	Функция integral()	Выполнено
5	Режим тестирования (-t)	Выполнено

Пункт	Требование	Статус
6	Makefile с целями all и clean	Выполнено
7	Комментарии к глобальным и статическим переменным	Выполнено

Все 7 пунктов задания выполнены полностью.

6.2. Итоги по дополнительным возможностям

В программе реализованы **следующие возможности** сверх обязательных требований:

1. 4 метода поиска корней вместо одного
2. 3 метода интегрирования вместо одного
3. Выбор методов через опции `--root-method` и `--int-method`
4. Визуализация результатов через Gnuplot
5. Подробный вывод с опцией `-p`
6. Логирование в файл через `--log`
7. Детальная статистика итераций

СПИСОК ЛИТЕРАТУРЫ

1. Бахвалов Н.С., Жидков Н.П., Кобельков Г.М. Численные методы. – М.: Бином, 2015.
 2. Керниган Б., Ритчи Д. Язык программирования С. – М.: Вильямс, 2019.
 3. Самарский А.А., Гулин А.В. Численные методы. – М.: Наука, 1989.
 4. Калиткин Н.Н. Численные методы. – М.: Наука, 1978.
 5. Gnuplot Documentation. – <http://www.gnuplot.info/documentation.html>
 6. GeoGebra Manual. – <https://wiki.geogebra.org/en/Manual>
 7. Linux Programmer's Manual. – man7.org/linux/man-pages
-

Программа разработана и протестирована в ОС Linux (Mint, Cinnamon).

Дата выполнения: 2026 г.